

Model-Adaptive Tool Necessity Reveals the Knowing-Doing Gap in LLM Tool Use

Yize Cheng* Chenrui Fan* Mahdi JafariRaviz* Keivan Rezaei Soheil Feizi
University of Maryland, College Park
{yzcheng, cfan42, krezaei, mahdij, sfeizi}@umd.edu

🔗 Project: <https://github.com/chengez/Tool-Cognition-Action>

Abstract

Large language models (LLMs) increasingly act as autonomous agents that must decide when to answer directly vs. when to invoke external tools. Prior work studying adaptive tool use has largely treated tool necessity as a model-agnostic property, annotated by human or LLM judge, and mostly cover cases where the answer is obvious (e.g., fetching the weather vs. paraphrasing text). However, tool necessity in the wild is more nuanced due to the divergence of capability boundaries across models: a problem solvable by a strong model on its own may still require tools for a weaker one. In this work, we introduce a model-adaptive definition of tool-necessity, grounded in each model’s empirical performance. Following this definition, we compare the necessity against observed tool-call behavior across four models on arithmetic and factual QA dataset, and find substantial mismatches of 26.5–54.0% and 30.8–41.8%, respectively. To diagnose the failure, we decompose tool use into two stages: an internal cognition stage that reflects whether a model believes a tool is necessary, and an execution stage that determines whether the model actually makes a tool-call action. By probing the LLM hidden states, we find that both signals are often linearly decodable, yet their probe directions become nearly orthogonal in the late-layer, last-token regime that drives the next-token action. By tracing the trajectory of samples in the two-stage process, we further discover that the majority of mismatch is concentrated in the cognition-to-action transition, not in cognition itself. These results reveal a *knowing–doing gap* in LLM tool-use: improving tool-use reliability requires not only better recognition of when tools are needed, but also better translation of that recognition into action.

1 Introduction

Large language models (LLMs) are increasingly deployed as autonomous agents that interact with external tools such as search engines, calculators, and APIs [20, 24, 26, 19]. A central challenge in building reliable autonomous LLM agents is achieving adaptive tool using: the LLM needs to determine *when* it should rely on such tools versus answering directly [8, 22, 27]. Prior work studying adaptive tool use [8, 22, 13] has largely treated tool necessity as a static, model-agnostic property, typically relying on human annotators or strong LLM judges to determine whether a query requires a tool, focusing primarily on polarized cases where the answer is obvious, such as fetching real-time weather data versus paraphrasing a static paragraph. However, tool necessity in the wild is fundamentally more nuanced due to the natural divergence of capability boundaries across different models. A problem that is easily solvable by a state-of-the-art model relying solely on its internal weights may completely exceed the capabilities of a smaller or less capable model, thereby making tool use strictly necessary for the latter but redundant for the former.

*Equal contribution

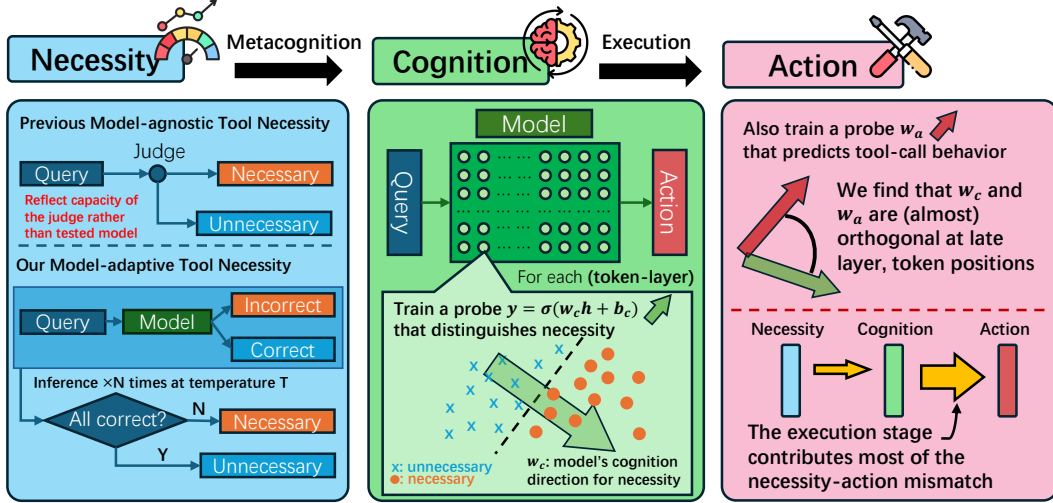


Figure 1: **Overview of the two stage cognition-execution modeling of LLM tool-use.** (Left) **Necessity**: We introduce a model-adaptive definition of tool necessity based on a model’s empirical ability to consistently answer a query correctly on its own, contrasting with prior model-agnostic approaches. (Middle) **Cognition**: By probing the model’s internal hidden states h , we identify a linear cognition direction w_c that successfully distinguishes when a tool is necessary. (Right) **Action**: We also train a probe w_a to predict the actual tool-call execution. We find that w_c and w_a become nearly orthogonal in late layers, and that the majority of the necessity-action mismatch stems from the execution stage (translating awareness into action) rather than the internal cognition stage.

In this work, we argue that tool necessity must be intrinsically tied to the specific capabilities of the model in question. We introduce a model-adaptive definition of tool necessity, grounded not in static annotations, but in each individual model’s empirical performance. By evaluating necessity relative to a model’s intrinsic capabilities, we establish a more accurate characterization for when a specific LLM should seek external help. Following this definition, we compare the actual necessity against the observed tool-call behavior across four distinct models on arithmetic and factual question-answering (QA) datasets. Our findings reveal substantial mismatches: models exhibit a 26.5–54.0% necessity-action mismatch in arithmetic tasks and a 30.8–41.8% necessity-action mismatch in factual QA, frequently calling tools when capable of answering directly, or attempting to answer directly when lacking the requisite internal knowledge.

To diagnose the underlying mechanisms of this failure, we propose a two-stage decomposition of the tool-use process: an internal cognition stage, which reflects whether the model’s internal representations encode the belief that a tool is necessary, and an execution stage, which determines whether the model actually outputs the tool-triggering tokens. Building on prior advancements in mechanistic interpretability and representation engineering [35] and following recent literature on adaptive tool-using [13, 28], we probe the LLM hidden states and find that both the cognition of necessity and the execution intent are often linearly decodable. Yet, intriguingly, their respective probe directions become nearly orthogonal in the late-layer, last-token regime.

By tracing the trajectory of samples through this two-stage process, we uncover a *knowing-doing gap* in LLM tool use: the majority of the observed necessity-action mismatch cases originates from the transition from cognition to action, rather than in the cognition stage. Models frequently generate internal representations indicating the awareness of their own limitations, but fail to translate this into the syntactic execution of a tool call. Our main contributions can be summarized as follows:

- We introduce a model-adaptive definition of tool necessity grounded in empirical performance, challenging the traditional reliance on static, model-agnostic annotations.
- We evaluate four distinct LLMs across arithmetic and factual QA datasets, revealing substantial behavioral mismatches (up to 54.0%) between actual tool necessity and observed tool-call actions.
- By dividing tool use into an internal cognition stage and an execution stage, we use representation probing to demonstrate that while both intent and necessity are linearly decodable, their probe directions become near orthogonal in the late-layer, last-token regime.

- Through trajectory tracing, we discover that tool-use failures predominantly occur during the transition from cognition to action, highlighting a knowing-doing gap in LLM tool-use.

2 Related work

Tool calling in LLM agents. To extend LLM capabilities beyond parametric knowledge, researchers have introduced function/tool calling [20, 24, 26, 19], enabling interaction with external resources and expanding task coverage. Standardized protocols like MCP [1] and A2A [6] further streamline communication and access within tool ecosystems. In parallel, various works have examined tool-use accuracy [12, 21], hallucinated calls [33, 23], and robustness to tool descriptions [25, 5]. However, while these efforts aim at teaching and evaluating *how* to use tools, an important and often understudied challenge in building reliable LLM agents is determining *when* to use tools. Existing works that do study this challenge [8, 22, 13] treat tool necessity as a static property of the query, labeling instances as either tool-necessary or tool-unnecessary using human annotators or some proprietary LLM. This ignores the inherent difference in capability boundaries between different models. While Wang et al. [27] has also advocated for model-dependent tool necessity, to the best of our knowledge, we are the first to have a pipeline that empirically grounds tool necessity in the actual capabilities of a given model.

Meta-cognition of LLMs and the “knowing-doing gap”. The ability of LLMs to accurately assess their own capability boundaries—often referred to as meta-cognition or self-assessment—has been a topic of long-standing interest [10, 30]. To measure this self-awareness, early work primarily relies on measuring explicit self-assessment by teaching models to express their knowledge boundaries [2, 31] or to directly verbalize confidence [15]. However, recent work has shown that the ability for models to verbalize its internal activations is limited [17, 9]. Moreover, the task of self-assessment and actual problem solving are fundamentally different tasks. When explicitly prompted about its capability boundary, the model would focus on self-assessment. But when faced with actual problem solving, the prompt is usually tasks-oriented, and hence the self-assessing process becomes implicit and subconscious. This akin to the distinction between system I and system II thinking [14]. Therefore, in this work, we follow some recent work that use internal state probing to measure models’ cognition of tool-necessity [13, 28], and also empirically show in Appendix B how model tool-call actions change when explicitly prompted for self-assessment.

Meanwhile, papers in other domain of LLMs that leverage hidden states to study model internal cognition have found that the model’s action can diverge from its internal belief. For example, Zhao et al. [34] find that LLMs may fail to refuse harmful queries despite internally recognizing their harmfulness, and Zhang et al. [32] show that models can internally recognize their inability to solve certain math problems yet still expend tokens on unproductive reasoning. In this work, we show that this “knowing-doing gap” similarly exists in tool-calling, and it can constitute even a larger proportion of end-to-end errors.

3 Defining model-adaptive tool necessity and two-stage modeling of tool-call

To study tool-use behavior in LLMs, we introduce a simple decomposition that separates recognizing the need for a tool from acting on that recognition. This distinction will serve as the foundation for the evaluation, diagnosis, and analysis throughout the rest of this paper.

Defining model-adaptive tool necessity. Existing work typically assumes a fixed notion of tool necessity, assigning each query a static label independent of the model being evaluated. However, we argue that since different models have different capability boundaries, the tool necessity label should be adaptive according to the model. To characterize a model’s capability boundary, given a model f and query x , we perform N independent inference runs without access to external tools at temperature T . If the model f can consistently solve the problem x correctly across N runs, we assume that this x falls within the f ’s capability boundary and therefore the tool necessity, $n_f(x)$, is 0. Otherwise, the model cannot reliably solve this query, and hence $n_f(x)$ is 1. The parameters N and T control the strictness of this criterion. Specifically, larger values of N and T yield a more conservative and robust estimate of whether a query truly falls within the model’s capability boundary as they demand the model to output the correct answer more consistently.

This formulation captures a key aspect of real-world deployment: reliability under uncertainty. In practical settings, a model that only occasionally produces the correct answer without tools may still benefit from external assistance to ensure consistent performance. By grounding tool necessity in empirical behavior rather than static annotation, our approach provides a more faithful characterization of when tool use is genuinely required for a given model.

The cognition-execution modeling of tool-call. We conceptualize tool use as a two-stage process:

$$x \rightarrow z_f(x) \rightarrow a_f(z_f(x)), \quad (1)$$

where $z_f(x)$ represents the model’s internal cognition of whether a tool is needed, and $a_f(z_f(x))$ denotes whether the model actually invokes a tool, based on its cognition. This two-stage decomposition mirrors the cognition process of human and what we desire for the model. It distinguishes between *meta-cognition*—the model’s internal belief about its capability boundary, and *execution ability*—how model acts based on its cognition.

End-to-end error diagnosis. Under our model-dependent definition of tool necessity $n_f(x)$ and the two stage modeling as in Equation 1, we can decompose the end-to-end necessity-action mismatch, $D(n_f(x), a_f(z_f(x)))$, into the mismatch between actual necessity and cognition $D(n_f(x), z_f(x))$, and the mismatch between model’s cognition and actual decision $D(z_f(x), a_f(z_f(x)))$, where $D(m, n)$ denotes the discrepancy between m and n .

4 Dataset curation

We cover two representative domains: math arithmetic and factual question answering, using two widely used model families: Qwen3-8B and Qwen3-4B [29], as well as Llama-3.1-8B-Instruct and Llama-3.2-3B-Instruct [7]. These domains provide natural testbeds in which some queries can be reliably solved by the model alone, while others may require external assistance (i.e. a calculator for arithmetic tasks and a search API for factual queries). For math arithmetic dataset, we mix problem types that vary in both surface form and actual difficulty. It includes simple one- and two-step addition and subtraction problems, along with harder examples involving multi-digit multiplication, modulo, parentheses, operator precedence, and longer addition/subtraction chains, resulting in a total of 4,000 instances. This gives us problems with a range of difficulty levels from very simple questions to extremely difficult ones, enabling us to measure the capability boundary of the model. More details about the curation of our arithmetic dataset can be found in Appendix A. For factual question answering, we adopt TruthfulQA [16], a widely used dataset with 817 instances designed to evaluate the factual reliability of language models.

4.1 Grounding tool necessity to model-specific capability boundaries

We follow our definition in Section 3 and run $N = 10$ independent inferences at temperature $T = 0.7$ without access to external tools. For a specific model, we count samples where the model fails at least once as *tool-necessary*, and samples where the model consistently gives correct answers across all $N = 10$ runs as *tool-unnecessary*. Figure 2 shows that different models have substantially different capability boundaries, which would be obscured by the model-agnostic definition of tool necessity. Specifically, the clean boundary in the first row is induced by our sorting procedure, while the red-green disagreements across rows show that the same sample groups can fall on different sides of different models’ capability boundaries. This pattern appears in both arithmetic and factual question answering, suggesting that tool necessity depends not only on task type or dataset membership, but also on the particular model being deployed. This motivates using $n_f(x)$ rather than a single global necessity label when evaluating tool-use judgment and downstream call behavior.

4.2 Collecting tool-call behaviors on *tool-necessary* and *tool-unnecessary* instances

We run inference on the LLMs using both the *tool-necessary* and *tool-unnecessary* instances obtained in Section 4.1. In this setting, models are provided access to external tools: a calculator for arithmetic question answering and a search API for factual queries. To facilitate the diagnostic interpretation efforts in Section 5, greedy decoding is used when collecting tool-call actions. To better reflect real-world deployment, we follow existing practice [21, 3] and implement model-specific handlers

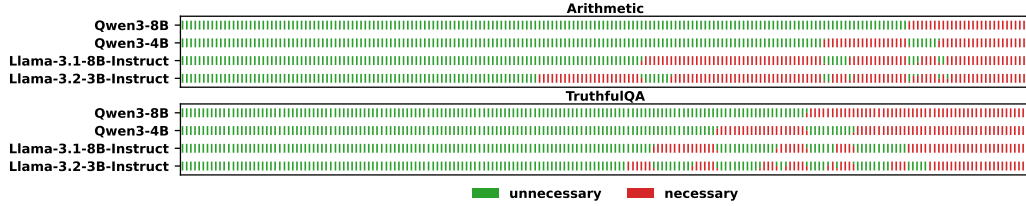


Figure 2: **Model-dependent tool-call necessity.** Each vertical bar represents 0.5% of samples. Green indicates samples answered correctly in all $N = 10$ no-tool runs; red indicates at least one failure. Within each dataset, samples share the same order across rows, obtained by recursively sorting within each previous model’s correctness partition.

that expose these tools in the syntax expected by each model. We then further divide *tool-necessary* and *tool-unnecessary* samples based on the model’s actual tool-call behavior, and obtain 4 sets of data: *Necessary-Called* (N-C), *Necessary-NotCalled* (N-NC), *Unnecessary-Called* (UN-C), and *Unnecessary-NotCalled* (UN-NC). The first and last are aligned with the optimal behavior under our model-dependent definition of necessity, while the middle two correspond to the end-to-end necessity–action mismatch $D(n_f(x), a_f(z_f(x)))$ defined in Section 3.

Table 1: Breakdown of tool-call behavior across the four categories defined by the model-dependent necessity $n_f(x)$ and the observed action. Aligned cells (N-C: Necessary-Called, UN-NC: Unnecessary-NotCalled) are shaded green; misaligned cells (N-NC, UN-C) are shaded red and together form the end-to-end mismatch, summarized in the gray **Mis.** column.

Model	Arithmetic					TruthfulQA				
	N-C	N-NC	UN-C	UN-NC	Mis.	N-C	N-NC	UN-C	UN-NC	Mis.
Qwen3-8B	438 (11.0%)	140 (3.5%)	1526 (38.2%)	1896 (47.4%)	41.7%	69 (8.4%)	146 (17.9%)	108 (13.2%)	494 (60.5%)	31.1%
Qwen3-4B	253 (6.3%)	581 (14.5%)	481 (12.0%)	2685 (67.1%)	26.5%	103 (12.6%)	153 (18.7%)	189 (23.1%)	372 (45.5%)	41.8%
Llama-3.1-8B-Instruct	419 (10.5%)	1204 (30.1%)	335 (8.4%)	2042 (51.1%)	38.5%	98 (12.0%)	130 (15.9%)	122 (14.9%)	467 (57.2%)	30.8%
Llama-3.2-3B-Instruct	526 (13.2%)	1559 (39.0%)	600 (15.0%)	1315 (32.9%)	54.0%	58 (7.1%)	164 (20.1%)	104 (12.7%)	491 (60.1%)	32.8%

End-to-end mismatch is substantial. Table 1 reports the distribution of the four categories across four models and two domains. The aggregated mismatch rate (gray **Mis.** column) ranges from **26.5%** to **54.0%** on arithmetic and from **30.8%** to **41.8%** on TruthfulQA, indicating that with a model-specific notion of tool necessity, between roughly one quarter and one half of all queries result in a tool-use action that is inconsistent with the model’s actual capability. This mismatch rate between actual tool necessity and model tool-use action further highlights the importance of determining *when* to use tools, an issue that is often overlooked in prior work that only emphasizes *how* to use them.

The dominant failure mode is highly model- and domain-dependent. Beyond the overall mismatch rates, the specific types of errors vary significantly across both models and domains. On arithmetic, Qwen3-8B suffers from tool-overuse (UN-C at **38.2%** vs. N-NC at **3.5%**). In contrast, Qwen3-4B and both Llama models exhibit clear tool underuse, with N-NC rates of **14.5%** (Qwen3-4B), **30.1%** (Llama-3.1-8B-Instruct), and **39.0%** (Llama-3.2-3B-Instruct), exceeding their respective UN-C rates. Interestingly, these tendencies are not consistent even within a single model. On TruthfulQA, Qwen3-8B reverses its trend entirely, showing tool underuse (N-NC at **17.9%** vs. UN-C at **13.2%**), while Qwen3-4B now shows tool-overuse (UN-C at **23.1%** vs. N-NC at **18.7%**). Because these models shift between being overly eager and overly conservative in tool-calling depending on the context, it is clear that no single, uniform bias can fully explain these mismatch errors. Therefore, in the next section, we leverage our two-stage modeling of LLM tool-use defined in Section 3 for more fine-grained diagnosis.

5 From meta-cognition to execution ability: What went wrong?

Having measured the model-dependent tool necessities for each model (i.e., their capability boundaries) and collected their actual tool-call behaviors, we now examine where the breakdown between actual necessity and final action occurs, following the two-stage decomposition in Section 3. We first show that each stage—the internal cognition of necessity, and the executed action—is individually linearly separable from the model’s hidden states (Section 5.1 and Section 5.2), and then characterize the geometric relationship between the two (Section 5.3). Finally, we find that the majority of the error originates in the execution stage through per sample tracing (Section 5.4).

5.1 Probing for model’s cognition

Linear probing is a standard method for studying how concepts are represented in a model’s hidden-state space. Recent works [13, 28] have used it as a proxy for models’ internal belief of tool necessity and reported that, despite substantial end-to-end mismatch, the hidden states of *tool-necessary* and *tool-unnecessary* samples are *almost linearly separable*. Because that conclusion was drawn under a static, query-only definition of tool necessity, it is unclear whether it survives the model-dependent definition introduced in Section 3, where the necessity label $n_f(x)$ varies across models with different capability boundaries.

Concretely, we train a linear classifier with weight $\mathbf{w}_c$ and bias b_c on the model’s hidden states, using a learning rate of 0.01 with the Adam [11] optimizer, minimizing the following objective:

$$\mathcal{L} = -\frac{1}{K} \sum_{k=1}^K \left[n_f(x_k) \log \sigma(\mathbf{w}_c^\top h_t^{(l)}(x_k) + b_c) + (1 - n_f(x_k)) \log(1 - \sigma(\mathbf{w}_c^\top h_t^{(l)}(x_k) + b_c)) \right], \quad (2)$$

where x_k is a sample in the dataset and $h_t^{(l)}(x_k)$ is the hidden state at token position t and layer l . $\mathbf{w}_c$ also serves as the normal vector of the separating hyperplane, indicating the direction from “unnecessary” to “necessary” in the model’s representation space. We sweep (t, l) over all layers and over the last 20 query tokens; negative indices denote token positions relative to the start of generation, e.g., $t = -1$ is the final query token. As the class distribution is imbalanced (Table 1), we report the probe performance using the Matthews Correlation Coefficient (MCC) [18] on the held-out test set (30% of data), which is a more robust metric than accuracy or F1 under skewed labels:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (3)$$

Typically, an MCC value between 0.3-0.5 is considered moderate to good performance, and an MCC of 0.5 or more is considered good to strong performance. Figure 3 shows the MCC of probes trained at each (t, l) position for all four models on Arithmetic and TruthfulQA.

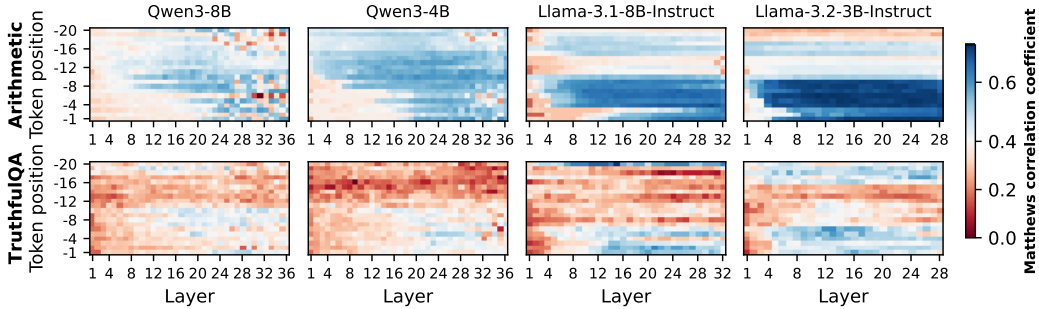


Figure 3: **Necessity probe performance across token-layer positions.** Each cell reports the held-out MCC of a linear probe trained to predict the model-adaptive necessity from the hidden state at a given layer and token position. Darker blue indicates stronger linear separability. The linear separability is strongly task-dependent, and the heatmap structure appears similar within model families.

Linear separability of necessity is strongly task-dependent. Under our model-adaptive definition, the prior “almost linearly separable” picture partially holds. On Arithmetic, necessity is linearly separable for most models, with broad regions of mid-to-late layers crossing $\text{MCC} = 0.4$. This aligns with the finding in prior works [13, 28]. On TruthfulQA, however, the regions where MCC exceeds 0.4 is noticeably smaller, with only near-last tokens in mid-late layers of Llama models still display decent separability. This contrast suggests the challenge in distinguishing model-adaptive *tool-necessary* and *tool-unnecessary* samples, which is more nuanced than the obvious cases prior work focus on [8, 13, 28]. It also suggests that tool-necessity signals are easier to surface in tasks where problem difficulty is reflected in the input’s surface structure, such as arithmetic, where complexity grows with the expression itself. In open-domain factual QA, however, surface form provides little cue about underlying difficulty, making tool necessity or epistemic uncertainty harder to linearly separate. The heatmap structure also appears similar within model families, with two Qwen and two Llama models sharing similar patterns respectively.

Decent internal signal coexists with large end-to-end mismatch. The probe reaches decent MCC at many (t, l) positions, indicating that information about the model’s capability boundary is in fact present in the residual stream. Yet the same models still exhibit substantial end-to-end necessity–action mismatch (Table 1), meaning this internal signal is not effectively converted into the right tool-call decision at generation time. This mismatch between “what the hidden states know” and “what the model does” is a first hint of a knowing–doing gap, and motivates the next two questions: does the model encode its action in a similarly separable way (Section 5.2), and how does the action representation relate to the cognition representation (Section 5.3)?

5.2 Probing for action

Having characterized how necessity is represented internally, we now ask the parallel question for the model’s actual decision: how linearly separable is the executed action—whether the model invokes a tool or not—from the same hidden states? Concretely, we train a linear classifier $(\mathbf{w}_a, b_a)$ with the same objective as in Equation 2 while just changing $(\mathbf{w}_c, b_c)$ to $(\mathbf{w}_a, b_a)$. The Probe performance on the held-out test set (30% of data) in terms of MCC is shown in Figure 4.

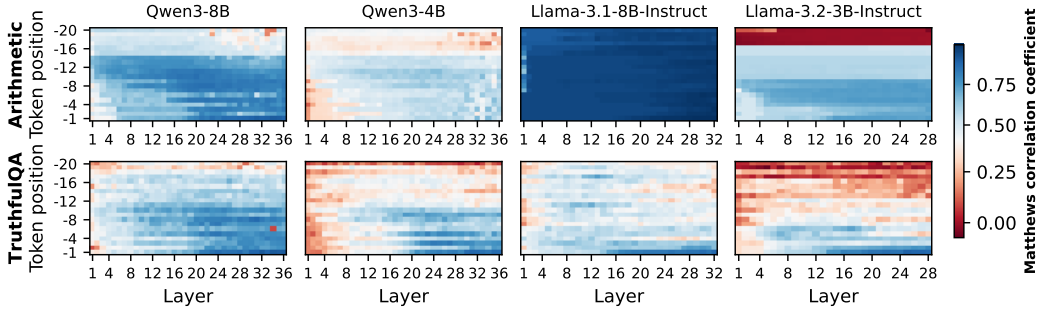


Figure 4: **The action Probe performance on different position in Matthews correlation coefficient.** Each cell reports the held-out MCC of a linear probe trained to predict the tool-call action from the hidden state at a given layer and token position. Darker blue indicates stronger linear separability. The action signal appears highly linearly separable in the hidden states, particularly in near-end tokens and late layers.

The action is highly separable from hidden states. Figure 4 shows that, on both Arithmetic and TruthfulQA dataset, the action probe attains $\text{MCC} \geq 0.4$ over broad regions of nearly every model. The signal spans most layers and token positions rather than being confined to a narrow band, indicating that whether the model is about to call a tool is a strongly decodable feature from its residual stream, aligning with recent finding [4].

5.3 The gap between cognition and execution

The two probes give us, at every (t, l) , a pair of direction vectors: $\mathbf{w}_c$ pointing from “unnecessary” to “necessary” in the model’s representation space, and $\mathbf{w}_a$ pointing from “no-call” to “call.” If tool-use behavior were a direct readout of the model’s internal necessity assessment, the two directions should align—at least in the layers where both probes succeed. We test this by computing the cosine similarity $\text{CosSim}(\mathbf{w}_c, \mathbf{w}_a)$ between $\mathbf{w}_c$ and $\mathbf{w}_a$ at each position: a value near ± 1 means necessity and action are encoded along (anti-)parallel directions, while a value near 0 means the two are represented in geometrically independent subspaces.

Partial alignment between $\mathbf{w}_c$ and $\mathbf{w}_a$ exists in intermediate token-layer positions. Figure 5 shows that the cosine similarity between $\mathbf{w}_c$ and $\mathbf{w}_a$ is fairly high in notable regions of several heatmaps, particularly covering a sizable area for the two Qwen models on Arithmetic. So necessity and action are not encoded in entirely disjoint subspaces: in some intermediate token-layer positions, the directions share meaningful alignment.

Alignment collapses at the position that drives generation. The picture changes sharply at the position that actually determines the next token: the late layers of the final query token ($t = -1$, large l). For the two models with the strongest mid-stream alignment, Qwen3-8B and Qwen3-4B, the cosine similarity falls back to small values exactly in the bottom right corner of the heatmap, so $\mathbf{w}_c$

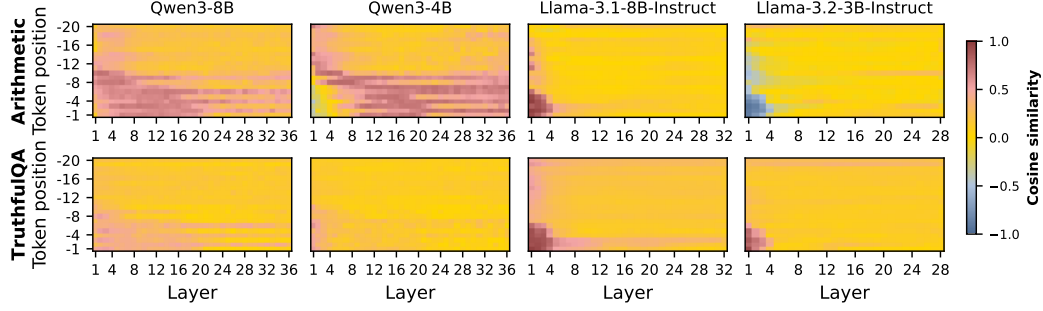


Figure 5: **The cosine similarity score between w_c and w_a on different positions.** The similarity scores between two probe direction are small across the majority of the area. Although for some models there are moderate similarity scores in late token and middle layer position, two directions fall back to near orthogonal relationship in the late layer of the last token (bottom right corner).

and w_a become close to orthogonal precisely where they would need to interact to translate “I should call a tool” into the actual call token. The same trend toward low cosine at late layers / last token holds, more uniformly, for the other models and for TruthfulQA. Whatever partial coupling exists in earlier layers therefore does not survive to the readout.

The previous two subsections established that the model’s hidden states often contain a usable necessity signal yet still produce mismatched tool-call actions. Figure 5 explains *why*: even when necessity and action share some structure in intermediate representations, the two directions become nearly orthogonal in the late-layer / last-token regime that ultimately drives the next-token decision.

5.4 Two stage error diagnosis and attribution

So far we have established two facts: end-to-end necessity–action mismatch is substantial (Table 1), and the cognition and action directions are nearly orthogonal at the readout (Section 5.3). However, the results in Section 5.3 tells us only that the two stages are *decoupled*, not which of them is responsible for the mismatch we observe. To attribute the error, we trace each sample along the *Factual* $\rightarrow$ *Cognition* $\rightarrow$ *Action* modeling, taking Cognition to be the necessity probe (w_c, b_c) read out at the last query token and last layer—the same position that drives the next-token decision. Each sample then falls into one of four categories: correct in both stages (green), stage-one-only error (red), stage-two-only error (orange, the knowing–doing gap), or compensating errors that cancel at the action (purple). We show the full Sankey flow diagram in Figure 6.

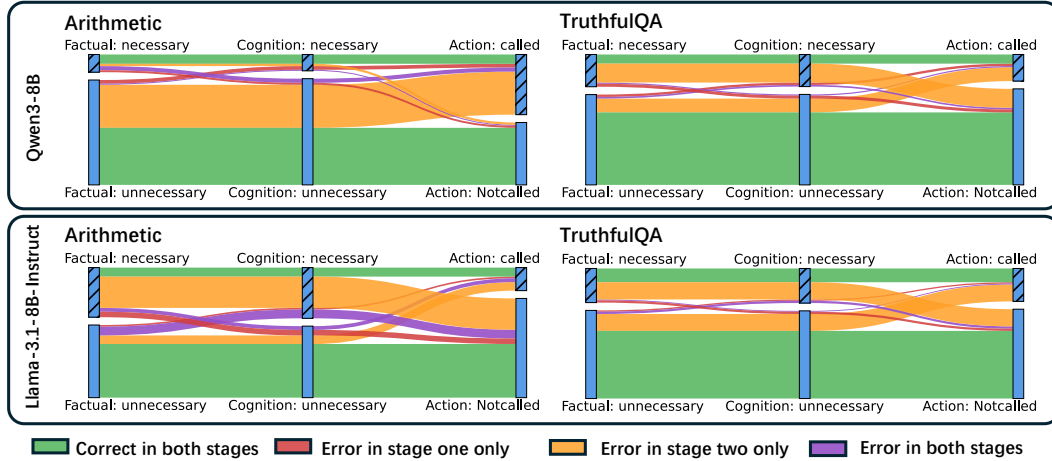


Figure 6: **Per-sample two-stage decomposition of tool-call behavior on Arithmetic and TruthfulQA, for Qwen3-8B (top) and Llama-3.1-8B-Instruct (bottom).** Each flow tracks a sample through three nodes: ground-truth necessity (*Factual*), the model’s internal cognition of necessity (*Cognition*), and the executed action (*Action*). The end-to-end error is dominated by **orange** flow, where cognition is correct but action flips away from it—the knowing–doing gap.

Stage two carries the majority of error. In all four panels of Figure 6, the orange flow (samples with stage two error only) is by far the largest error category, while red (samples with stage one error only) is rather thin. Given that cognition and action are decoupled, the asymmetric orange $\gg$ red localizes the failure: the end-to-end mismatch is overwhelmingly produced in the cognition $\rightarrow$ action stage rather than in forming cognition itself. The bottleneck is therefore not knowing whether a tool is needed, but converting that knowledge into the call/no-call action. This shows that making correct *when* to use tools decisions is not just about having the correct tool-necessity cognition, but more importantly also about translating that cognition to actual matching action.

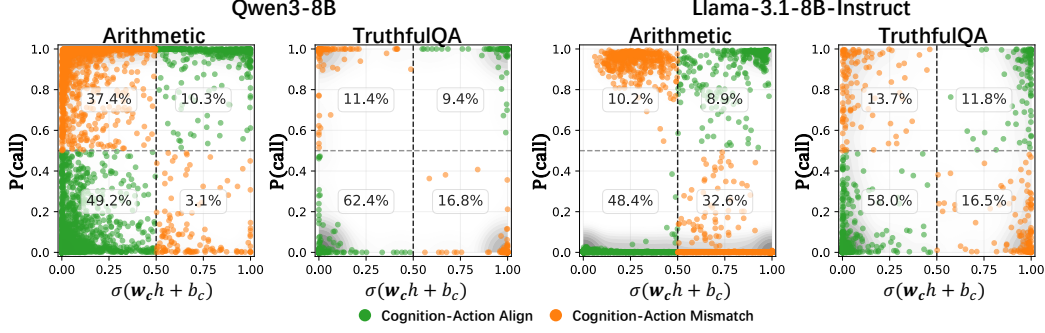


Figure 7: **The confidence of cognition versus tool calling behavior.** The x axis denotes the probe output after sigmoid function, and the y axis represents the tool call probability quantified by Equation 4. The mismatch can persist even when the internal representation strongly indicates that a sample is either *tool-necessary* or *tool-unnecessary*.

The cognition–execution mismatch is not associated with cognition confidence. Given the substantial gap between a model’s internal cognition and its final tool-call behavior, illustrated by the large orange band in Figure 6, a natural question is whether this mismatch is caused by uncertainty in the meta-cognitive belief itself. In other words, does the mismatch primarily occur on samples where the model is uncertain about whether a tool is necessary? To investigate this question, we quantify the confidence of meta-cognition using the post-sigmoid output of the cognition probe, $\sigma(\mathbf{w}_c h + b_c)$. We then plot, for all samples, the relationship between the “confidence of tool necessity” and the “probability of making a tool call” in Figure 7. The probability of making a tool call is defined as

$$P(\text{call}) = \frac{p(\langle \text{tool-token} \rangle)}{p(\langle \text{tool-token} \rangle) + p(\text{best non-tool token})}, \quad (4)$$

where $p(\cdot)$ denotes the softmax probability assigned by the language model to a candidate next token, $\langle \text{tool-token} \rangle$ denotes the model-specific token that initiates a tool call (e.g., `<tool_call>` in Qwen models), and the “best non-tool token” refers to the highest-logit token among all tokens that are not tool-call tokens. This formulation nicely normalizes the probability to the range $[0, 1]$. Since greedy decoding is used when collecting tool-call behaviors (Section 4.2), a value of $P(\text{call}) > 0.5$ corresponds to an actual tool call being generated. As shown in Figure 7, the cognition–execution mismatch does not primarily occur near the uncertain region where $\sigma(\mathbf{w}_c h + b_c) \approx 0.5$. Instead, many orange points occur in regions where $\sigma(\mathbf{w}_c h + b_c)$ is close to 0 or 1. This observation suggests that the cognition–execution mismatch is not driven by low confidence in the model’s internal cognition. Rather, the mismatch can persist even when the internal representation strongly indicates that a sample is either *tool-necessary* or *tool-unnecessary*.

6 Conclusion

In this work, we introduced a model-adaptive definition of tool necessity that grounds evaluation in empirical capabilities, and revealed a substantial mismatch between when models actually need tools and when they invoke them. By decomposing the tool-use process into internal cognition and execution stages, and analyzing hidden state representations, we identified a fundamental “knowing-doing gap” in LLMs. While models sometimes internally recognize the tool necessity, these cognitive representations become orthogonally misaligned with execution intent in later layers, leading to failures in taking the appropriate action. Our findings demonstrate that improving autonomous agents requires not just better internal meta-cognition, but bridging the knowing-doing gap to ensure self-awareness translates into reliable execution.

Acknowledgments

This work was supported in part by NSF CAREER Award 1942230, the ONR PECASE Award N00014-25-1-2378, ARO Early Career Program Award 310902-00001, Army Grant W911NF-21-2-0076, NSF Award CCF-2212458, NSF Award 2229885 (NSF Institute for Trustworthy AI in Law and Society, TRAILS), MURI Grant 14262683, DARPA AIQ Grant HR00112590066, and a Meta Research Award 314593-00001.

References

- [1] Anthropic. Introducing the model context protocol, November 2024. URL <https://www.anthropic.com/news/model-context-protocol>.
- [2] Lida Chen, Zujie Liang, Xintao Wang, Jiaqing Liang, Yanghua Xiao, Feng Wei, Jinglei Chen, Zhenghong Hao, Bing Han, and Wei Wang. Teaching large language models to express knowledge boundary from their own signals, 2024. URL <https://arxiv.org/abs/2406.10881>.
- [3] Yize Cheng, Arshia Soltani Moakhar, Chenrui Fan, Parsa Hosseini, Kazem Faghieh, Zahra Sodagar, Wenxiao Wang, and Soheil Feizi. Your llm agents are temporally blind: The misalignment between tool use decisions and human time perception, 2026. URL <https://arxiv.org/abs/2510.23853>.
- [4] Esakkivel Esakkiraja, Sai Rajeswar, Denis Akhiyarov, and Rajagopal Venkatesaramani. Therefore i am. i think, 2026. URL <https://arxiv.org/abs/2604.01202>.
- [5] Kazem Faghieh, Wenxiao Wang, Yize Cheng, Siddhant Bharti, Gaurang Sriramanan, Sriram Balasubramanian, Parsa Hosseini, and Soheil Feizi. Gaming tool preferences in agentic llms. *arXiv preprint arXiv:2505.18135*, 2025.
- [6] Google. Agent2agent (a2a) protocol. <https://google.github.io/A2A/>, 2025.
- [7] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [8] Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, and Lichao Sun. MetaTool benchmark for large language models: Deciding whether to use tools and which to use, 2024. URL <https://arxiv.org/abs/2310.03128>.
- [9] Li Ji-An, Hua-Dong Xiong, Robert C. Wilson, Marcelo G. Mattar, and Marcus K. Benna. Language models are capable of metacognitive monitoring and control of their internal activations, 2025. URL <https://arxiv.org/abs/2505.13763>.
- [10] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislaw Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know, 2022. URL <https://arxiv.org/abs/2207.05221>.
- [11] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.
- [12] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. *arXiv preprint arXiv:2304.08244*, 2023.

- [13] Wenjun Li, Dexun Li, Kuicai Dong, Cong Zhang, Hao Zhang, Weiwen Liu, Yasheng Wang, Ruiming Tang, and Yong Liu. Adaptive tool use in large language models with meta-cognition trigger. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13346–13370, 2025.
- [14] Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhiwei Li, Bao-Long Bi, Ling-Rui Mei, Junfeng Fang, Xiao Liang, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models, 2025. URL <https://arxiv.org/abs/2502.17419>.
- [15] Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching models to express their uncertainty in words, 2022. URL <https://arxiv.org/abs/2205.14334>.
- [16] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods, 2022. URL <https://arxiv.org/abs/2109.07958>.
- [17] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermy, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- [18] B.W. Matthews. Comparison of the predicted and observed secondary structure of t4 phage lysozyme. *Biochimica et Biophysica Acta (BBA) - Protein Structure*, 405(2):442–451, 1975. ISSN 0005-2795. doi: [https://doi.org/10.1016/0005-2795\(75\)90109-9](https://doi.org/10.1016/0005-2795(75)90109-9). URL <https://www.sciencedirect.com/science/article/pii/0005279575901099>.
- [19] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. Augmented language models: a survey. *arXiv preprint arXiv:2302.07842*, 2023.
- [20] Aaron Parisi, Yao Zhao, and Noah Fiedel. Talm: Tool augmented language models. *arXiv preprint arXiv:2205.12255*, 2022.
- [21] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [22] Cheng Qian, Emre Can Acikgoz, Hongru Wang, Xiusi Chen, Avirup Sil, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. SMART: Self-aware agent for tool overuse mitigation, 2025. URL <https://arxiv.org/abs/2502.11435>.
- [23] Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. When2call: When (not) to call tools. *arXiv preprint arXiv:2504.18851*, 2025.
- [24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36: 68539–68551, 2023.
- [25] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in llm agents. *arXiv preprint arXiv:2504.19793*, 2025.
- [26] Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, et al. Restgpt: Connecting large language models with real-world restful apis. *arXiv preprint arXiv:2306.06624*, 2023.
- [27] Hongru Wang, Cheng Qian, Manling Li, Jiahao Qiu, Boyang Xue, Mengdi Wang, Heng Ji, Amos Storkey, and Kam-Fai Wong. Position: Agent should invoke external tools only when epistemically necessary, 2026. URL <https://arxiv.org/abs/2506.00886>.

- [28] Youjin Wang, Run Zhou, Rong Fu, Shuaishuai Cao, Hongwei Zeng, Jiaxuan Lu, Sicheng Fan, Jiaqiao Zhao, and Liangming Pan. ASA: Training-free representation engineering for tool-calling agents, 2026. URL <https://arxiv.org/abs/2602.04935>.
- [29] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [30] Zhangyue Yin, Qiushi Sun, Qipeng Guo, Jiawen Wu, Xipeng Qiu, and Xuanjing Huang. Do large language models know what they don’t know? In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8653–8665, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.551. URL <https://aclanthology.org/2023.findings-acl.551/>.
- [31] Hanning Zhang, Shizhe Diao, Yong Lin, Yi R. Fung, Qing Lian, Xingyao Wang, Yangyi Chen, Heng Ji, and Tong Zhang. R-tuning: Instructing large language models to say ‘i don’t know’, 2024. URL <https://arxiv.org/abs/2311.09677>.
- [32] Qingjie Zhang, Yujia Fu, Yang Wang, Liu Yan, Tao Wei, Ke Xu, Minlie Huang, and Han Qiu. Stop before you fail: Operational capability boundaries for mitigating unproductive reasoning in large reasoning models, 2026. URL <https://arxiv.org/abs/2509.24711>.
- [33] Yuxiang Zhang, Jing Chen, Junjie Wang, Yaxin Liu, Cheng Yang, Chufan Shi, Xinyu Zhu, Zihao Lin, Hanwen Wan, Yujiu Yang, et al. Toolbehonest: A multi-level hallucination diagnostic benchmark for tool-augmented large language models. *arXiv preprint arXiv:2406.20015*, 2024.
- [34] Jiachen Zhao, Jing Huang, Zhengxuan Wu, David Bau, and Weiyang Shi. LLMs encode harmfulness and refusal separately, 2025. URL <https://arxiv.org/abs/2507.11878>.
- [35] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to ai transparency, 2025. URL <https://arxiv.org/abs/2310.01405>.

A More details on arithmetic dataset curation

We generate math arithmetic problems grouped into three types. The easy group contains one-step addition/subtraction, short two-step chains, and small modulo problems. These give cases where a calculator is usually not needed. The larger short group keeps the expressions simple, but uses larger operands, including multi-digit subtraction, four-digit addition/subtraction, and two- or three-digit multiplication. These problems are still short enough to invite a direct answer, but they are more likely to cause digit errors. The multi-step group contains precedence chains, parenthesized expressions, multiplication chains, and long addition/subtraction chains. These examples test whether the model can track intermediate values and apply the order of operations, especially in cases that look simple but are easy to miscompute. We sample the dataset with a fixed random seed. During generation, we skip repeated expressions and resample until each family reaches its assigned sampling share. Table 2 gives the problem families and their sampling shares.

Table 2: Breakdown of the math arithmetic dataset. Shares show the fixed sampling share for each problem family. We use 4000 samples overall.

Group	Problem Family	Share	Detail	Example
Easy	Single-step arithmetic	8%	Two operands from 1–99 with an addition or subtraction operator.	$21 + 59$
	Two-step arithmetic	5%	Three operands from 1–99 with addition or subtraction operators.	$70 - 47 + 68$
	Small modulo	5%	Three-digit dividend with divisor from 3–19.	$562 \% 8$
Larger short	Negative subtraction	7%	Three- or four-digit operand minus a larger operand.	$390 - 554$
	Four-digit addition/subtraction	6%	Two four-digit operands.	$4921 - 9108$
	Two-digit multiplication	9%	Two two-digit operands.	$34 * 75$
	Three-by-two multiplication	9%	One three-digit factor and one two-digit.	$504 * 61$
	Three-by-three multiplication	7%	Two three-digit factors.	$867 * 671$
Multi-step	Precedence chain	12%	Five two- or three-digit operands with addition, subtraction, or multiplication operators.	$84 * 82 - 755 - 805 - 29$
	One-digit addition/subtraction chain	11%	16–39 one-digit terms with addition or subtraction operators.	$3 + 4 + 1 + 3 - 8 - \dots$
	Small addition/subtraction chain	10%	21–27 terms from 1–30 with addition or subtraction operators.	$9 + 5 - 1 - 3 + 23 + \dots$
	Parenthesized expression	6%	Four two-digit operands in $(a + b) \times (c - d)$ form.	$(67 + 68) * (52 - 88)$
	Multiplication chain	5%	Five two-digit operands in $a + b \times c - d \times e$ form.	$94 + 40 * 50 - 24 * 87$

Algorithms 1–13 specify the exact procedures used for generating the data samples in each family. In these algorithms, $\mathcal{U}\{m, \dots, n\}$ denotes the discrete uniform distribution over integers from m to n , and $\mathcal{U}(0, 1)$ denotes the continuous uniform distribution on the unit interval.

Algorithm 1 SINGLESTEPARITHMETIC

```

1:  $a, b \sim \mathcal{U}\{1, \dots, 99\}$ 
2:  $op \sim \{+, -\}$ 
3: return " $a \text{ } op \text{ } b$ "

```

Algorithm 2 TWOSTEPARITHMETIC

```

1:  $a, b, c \sim \mathcal{U}\{1, \dots, 99\}$ 
2:  $u \sim \mathcal{U}(0, 1)$ 
3: if  $u < 0.5$  then
4:   return " $a + b - c$ "
5: else
6:   return " $a - b + c$ "
7: end if

```

Algorithm 3 SMALLMODULO

```
1:  $a \sim \mathcal{U}\{100, \dots, 999\}$ 
2:  $b \sim \mathcal{U}\{3, \dots, 19\}$ 
3: return " $a \% b$ "
```

Algorithm 4 NEGATIVESUBTRACTION

```
1:  $u \sim \mathcal{U}(0, 1)$ 
2: if  $u < 0.55$  then
3:    $a \sim \mathcal{U}\{100, \dots, 500\}$ 
4:    $b \sim \mathcal{U}\{a + 10, \dots, a + 250\}$ 
5: else
6:    $a \sim \mathcal{U}\{1000, \dots, 5000\}$ 
7:    $b \sim \mathcal{U}\{a + 100, \dots, a + 3000\}$ 
8: end if
9: return " $a - b$ "
```

Algorithm 5 FOURDIGITADDITIONSUBTRACTION

```
1:  $a, b \sim \mathcal{U}\{1000, \dots, 9999\}$ 
2:  $u \sim \mathcal{U}(0, 1)$ 
3: if  $u < 0.6$  then
4:   return " $a + b$ "
5: else
6:   return " $a - b$ "
7: end if
```

Algorithm 6 TWODIGITMULTIPLICATION

```
1:  $u \sim \mathcal{U}(0, 1)$ 
2: if  $u < 0.45$  then
3:    $a, b \sim \mathcal{U}\{15, \dots, 50\}$ 
4: else
5:    $a, b \sim \mathcal{U}\{30, \dots, 99\}$ 
6: end if
7: return " $a \times b$ "
```

Algorithm 7 THREEBYTWOMULTIPLICATION

```
1:  $a \sim \mathcal{U}\{100, \dots, 999\}$ 
2:  $b \sim \mathcal{U}\{10, \dots, 99\}$ 
3: return " $a \times b$ "
```

Algorithm 8 THREEBYTHREEMULTIPLICATION

```
1:  $a, b \sim \mathcal{U}\{100, \dots, 999\}$ 
2: return " $a \times b$ "
```

Algorithm 9 PRECEDENCECHAIN

```
1:  $a_1, \dots, a_5 \sim \mathcal{U}\{10, \dots, 999\}$ 
2:  $op_1, \dots, op_4 \sim \{+, -, \times\}$ 
3: return " $a_1 op_1 a_2 op_2 a_3 op_3 a_4 op_4 a_5$ "
```

Algorithm 10 ONEDIGITADDITIONSUBTRACTIONCHAIN

```
1:  $u \sim \mathcal{U}(0, 1)$ 
2: if  $u < 0.4$  then
3:    $n \sim \mathcal{U}\{16, \dots, 22\}$ 
4: else
5:    $n \sim \mathcal{U}\{29, \dots, 39\}$ 
6: end if
7:  $a_1, \dots, a_n \sim \mathcal{U}\{1, \dots, 9\}$ 
8: For each  $i$ , sample  $op_i$  with  $\Pr(op_i = +) = 0.53$  and  $\Pr(op_i = -) = 0.47$ 
9: return " $a_1 op_1 a_2 op_2 \dots op_{n-1} a_n$ "
```

Algorithm 11 SMALLADDITIONSUBTRACTIONCHAIN

```
1:  $n \sim \mathcal{U}\{21, \dots, 27\}$ 
2:  $a_1, \dots, a_n \sim \mathcal{U}\{1, \dots, 30\}$ 
3:  $op_1, \dots, op_{n-1} \sim \{+, -\}$ 
4: return " $a_1 op_1 a_2 op_2 \dots op_{n-1} a_n$ "
```

Algorithm 12 PARENTHESESIZEDEXPRESSION

```
1:  $a, b, c, d \sim \mathcal{U}\{10, \dots, 99\}$ 
2: return " $(a + b) \times (c - d)$ "
```

Algorithm 13 MULTIPLICATIONCHAIN

```
1:  $a, b, c, d, e \sim \mathcal{U}\{10, \dots, 99\}$ 
2: return " $a + b \times c - d \times e$ "
```

B Explicitly prompting for verbalized belief of tool-necessity

Due to the limitation of LLMs in verbalizing internal decision processes [17, 9], and the fundamental difference between the task of self-assessment and actual problem solving, in this paper, we followed the approach in recent work that use internal state probing to measure models’ cognition of tool-necessity [13, 28]. Nevertheless, for completeness, we also report results obtained using explicit self-assessment prompts.

Specifically, we adopt a two-stage inference procedure. In the first stage, the model is given the same questions from the Arithmetic and TruthfulQA datasets, but instead of solving the problem directly, it is prompted to first decide “whether it is necessary to invoke an external tool” and to answer only with ‘yes’ or ‘no’. In the second stage, the model is instructed to “Now answer the original user request.”

Table 3 reports: (1) the MCC between the model’s ‘yes’/‘no’ responses and the actual capability-grounded tool necessity defined in Section 4.1; (2) the cognition–execution mismatch rate, defined as the proportion of samples where the model answered ‘yes’ but did not invoke a tool, or answered ‘no’ but eventually invoked one; and (3) the proportion of samples whose eventual tool-calling behavior changed relative to the direct task-oriented prompting setup used in Section 4.2.

The results show that the MCC of explicit ‘yes’/‘no’ judgments is substantially worse at capturing the actual capability-grounded notion of tool necessity. In particular, Llama-3.2-3B-Instruct achieves a negative MCC on TruthfulQA, while Llama-3.1-8B-Instruct simply answers ‘no’ for every TruthfulQA sample, resulting in an undefined MCC. This behavior implies that the model judges no sample to require a tool, which is clearly inconsistent with the capability measurements reported in Section 4.1. This poor MCC results further show the challenge in distinguishing model-adaptive tool-necessary and tool-unnecessary samples, which is more nuanced than the obvious cases prior work focus on [8, 13, 28].

In contrast, the cognition–execution mismatch rates are noticeably lower than those reported in Section 5.4. Llama-3.1-8B-Instruct even achieves a 0 mismatch rate, meaning that it not only answered ‘no’ for all samples, but also consistently refrained from making any tool calls. This outcome is somewhat expected: once the ‘yes’/‘no’ response becomes part of the model’s context, the model is more likely to remain consistent with that earlier commitment during subsequent generation.

Most importantly, however, Table 3 shows a large “Changed” rate in the third column of each dataset. Relative to direct problem solving with the task-oriented prompts used in our main experiments, explicit self-assessment changes tool-calling behavior on up to nearly 50% of samples. In practical deployments, prompts are typically task-oriented and designed to maximize task performance, rather than to elicit explicit self-assessment. Therefore, this substantial shift in behavior suggests that evaluations based on explicit prompts such as “decide whether it is necessary to invoke an external tool and answer ‘yes’ or ‘no’”, as used in some prior work [8, 13], may produce results that diverge significantly from models’ actual tool-use behavior under realistic task settings.

Table 3: Tool-call evaluation summary across datasets. For each model and dataset, we report Matthews Correlation Coefficient (MCC), mismatch rate (yes,false) + (no,true), and the proportion of changed tool-call behavior across variants.

Model	Arithmetic			TruthfulQA		
	MCC	Cog-Exe-Mis. (%)	Changed (%)	MCC	Cog-Exe-Mis. (%)	Changed (%)
Llama-3.1-8B-Instruct	0.0712	5.42	18.20	n/a	0.00	26.93
Llama-3.2-3B-Instruct	0.2251	19.62	29.18	-0.0400	9.67	27.05
Qwen3-4B	0.2330	3.85	29.55	0.1915	36.35	34.64
Qwen3-8B	0.2016	2.12	49.27	0.0538	13.34	20.93

C Limitations

In this paper, we instantiated the model-adaptive definition of tool necessity using $N = 10$ and $T = 0.7$ (as defined in Section 4.1). It could be beneficial to cover other instantiations of this definition with different N , T values to see how the necessity-action mismatch rate may change under different settings. Moreover, as an integral part of this work relies on probing model hidden states, this makes our work inapplicable to close-source state-of-the-art LLMs like GPT or Gemini.